Tarea M25-CD Avance Proyecto Parte II Javier Bueno Sanchez.

July 23, 2026

1 Tarea M25-CD Avance Proyecto Parte II Javier Bueno Sanchez.

Preámbulo.

Crearemos un clasificador para predecir si un nuevo paciente es propenso a un ataque al corazón, no sin antes hacer el Análisis Exploratorio previo correspondiente y no solo eso, también vas a predecir un número hecho en tu computador, para ellos utilizaremos la aplicación más potente de Windows (Exacto!!, Paint) y vas a predecir qué número es!!. La información necesaria la encontrarás en el repositorio de datos.

1.1 Supervised Learning

by Ivan Alducin

1.2 Índice.

1. Importación de librerías.
2. Análisis Exploratorio de Datos.
3. Algoritmo KNN.
4. Predicción.
5. Reconocimiento de dígitos.
6. Train/Test.
7. Reconocimiento de imagen.
8. Overfit and Underfit.
9. Regresión logística.
10. Conclusiones.

1. Importación de librerías.

[103]:
```
# Importación de librerías para lectura, análisis y visualización.
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

2. Análisis Exploratorio de Datos.

Vamos a trabajar con el conjunto de datos de Heart Attack, el objetivo es predecir bajo que escenario es más probable que un paciente pueda tener un ataque al corazón Un experto en medicina cardiovasuclar puede predecir esto sin hacer uso de Machine Learning, pero probablemente no instantáneamente, ¡y ciertamente no si estamos tratando con cientos o miles de muestras!.

A continuación una breve explicación de las variables del dataset:

- age: Age of the patient
- sex: Sex of the patient
- cp: Chest pain type ~ 0 = Typical Angina, 1 = Atypical Angina, 2 = Non-anginal Pain, 3 = Asymptomatic
- trtbps: Resting blood pressure (in mm Hg)
- chol: Cholestoral in mg/dl fetched via BMI sensor
- fbs: (fasting blood sugar > 120 mg/dl) ~ 1 = True, 0 = False
- restecg: Resting electrocardiographic results ~ 0 = Normal, 1 = ST-T wave normality, 2 = Left ventricular hypertrophy
- thalachh: Maximum heart rate achieved
- oldpeak: Previous peak
- slp: Slope
- caa: Number of major vessels
- thall: Thalium Stress Test result ~ (0,3)
- exng: Exercise induced angina ~ 1 = Yes, 0 = No
- output: Target variable

Nota técnica: Tratamiento según la naturaleza de las variables.

No todas las variables numéricas representan cantidades. Algunas utilizan números únicamente como códigos de categorías; por ello, aplicarles promedios, correlaciones o interpretaciones continuas puede producir conclusiones engañosas.

En este conjunto de datos se distinguirán:

Variables continuas: age, trtbps, chol, thalachh y oldpeak. Se analizarán mediante medias, medianas, dispersión y distribuciones.

Variables binarias: sex, fbs y exng. Se compararán mediante frecuencias y proporciones.

Variables categóricas codificadas: cp, restecg, slp, caa y thall. Se analizarán mediante tablas de contingencia y porcentajes por clase.

Variable objetivo: output. Antes de interpretarla, debe verificarse documentalmente qué representan los valores 0 y 1.

Que una variable esté almacenada como número no significa que sea cuantitativa. Su tratamiento debe depender de lo que representa, no solamente del tipo de dato que muestra Python.

[104]:
```
# Análisis exploratorio y validación de calidad.
df = pd.read_csv('/home/javi/Documentos/Tarea M25-CD Avance Proyecto Parte II␣
 ↪Javier Bueno Sanchez/Heart Attack.csv')

print('Dimensión original:', df.shape)
print('Duplicados:', df.duplicated().sum())
print('Valores nulos:', df.isna().sum().sum())
df = df.drop_duplicates().reset_index(drop=True)
print('Dimensión analítica:', df.shape)
display(df.head())
display(df.describe().T)
```

```
print('\nDistribución del objetivo:')
display(df['output'].value_counts().rename('Pacientes').to_frame())
```

```
Dimensión original: (303, 14)
Duplicados: 1
Valores nulos: 0
Dimensión analítica: (302, 14)
```

	age	sex	cp	trtbps	chol	fbs	restecg	thalachh	exng	oldpeak	slp	\
0	63	1	3	145	233	1	0	150	0	2.3	0	
1	37	1	2	130	250	0	1	187	0	3.5	0	
2	41	0	1	130	204	0	0	172	0	1.4	2	
3	56	1	1	120	236	0	1	178	0	0.8	2	
4	57	0	0	120	354	0	1	163	1	0.6	2	

	caa	thall	output
0	0	1	1
1	0	2	1
2	0	2	1
3	0	2	1
4	0	2	1

	count	mean	std	min	25%	50%	75%	max
age	302.0	54.420530	9.047970	29.0	48.00	55.5	61.00	77.0
sex	302.0	0.682119	0.466426	0.0	0.00	1.0	1.00	1.0
cp	302.0	0.963576	1.032044	0.0	0.00	1.0	2.00	3.0
trtbps	302.0	131.602649	17.563394	94.0	120.00	130.0	140.00	200.0
chol	302.0	246.500000	51.753489	126.0	211.00	240.5	274.75	564.0
fbs	302.0	0.149007	0.356686	0.0	0.00	0.0	0.00	1.0
restecg	302.0	0.526490	0.526027	0.0	0.00	1.0	1.00	2.0
thalachh	302.0	149.569536	22.903527	71.0	133.25	152.5	166.00	202.0
exng	302.0	0.327815	0.470196	0.0	0.00	0.0	1.00	1.0
oldpeak	302.0	1.043046	1.161452	0.0	0.00	0.8	1.60	6.2
slp	302.0	1.397351	0.616274	0.0	1.00	1.0	2.00	2.0
caa	302.0	0.718543	1.006748	0.0	0.00	0.0	1.00	4.0
thall	302.0	2.314570	0.613026	0.0	2.00	2.0	3.00	3.0
output	302.0	0.543046	0.498970	0.0	0.00	1.0	1.00	1.0

```
Distribución del objetivo:
```

output	Pacientes
1	164
0	138

[105]:
```
# Se muestran variables categóricas para distinguirlas de las continuas.
categorical_cols = ["sex", "cp", "fbs", "restecg", "slp", "caa", "thall"]
```

```python
# Se generan tablas con la distribución porcentual de cada columna para conocer más en profundidad los datos.
for col in categorical_cols:
    print(f"\nDistribución porcentual de {col}:")
    display(pd.crosstab(df[col], df["output"], normalize="columns").mul(100).round(2))
```

```
Distribución porcentual de sex:
```

output	0	1
sex		
0	17.39	43.9
1	82.61	56.1

```
Distribución porcentual de cp:
```

output	0	1
cp		
0	75.36	23.78
1	6.52	25.00
2	13.04	41.46
3	5.07	9.76

```
Distribución porcentual de fbs:
```

output	0	1
fbs		
0	84.06	85.98
1	15.94	14.02

```
Distribución porcentual de restecg:
```

output	0	1
restecg		
0	57.25	41.46
1	40.58	57.93
2	2.17	0.61

```
Distribución porcentual de slp:
```

output	0	1
slp		
0	8.70	5.49
1	65.94	29.88
2	25.36	64.63

```
Distribución porcentual de caa:
```

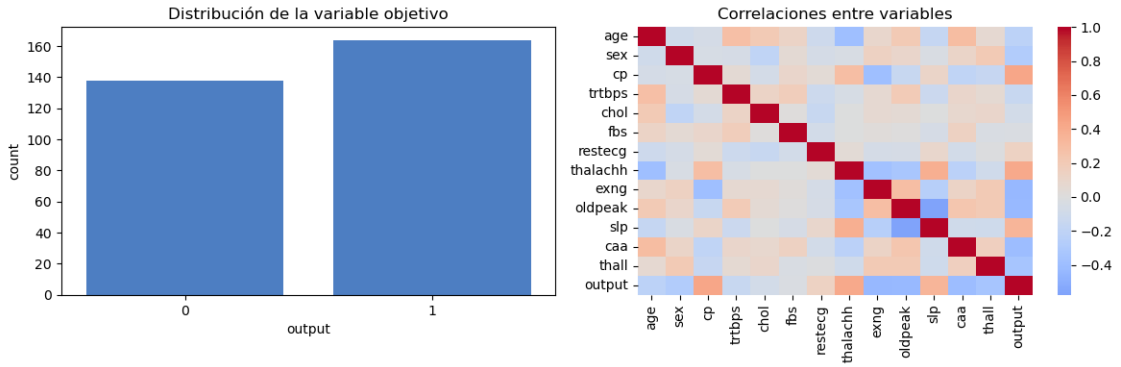
```
output      0      1
caa
0       32.61  79.27
1       31.88  12.80
2       22.46   4.27
3       12.32   1.83
4        0.72   1.83
```

Distribución porcentual de thall:

```
output      0      1
thall
0        0.72   0.61
1        8.70   3.66
2       26.09  78.66
3       64.49  17.07
```

[106]:
```
# Visualización de variables.
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
sns.countplot(data=df, x='output', ax=axes[0], color='#3A7BD5')
axes[0].set_title('Distribución de la variable objetivo')
sns.heatmap(df.corr(numeric_only=True), cmap='coolwarm', center=0, ax=axes[1])
axes[1].set_title('Correlaciones entre variables')
plt.tight_layout(); plt.show()

# Evidencia descriptiva: diferencias de medias, sin atribuir causalidad
group_means = df.groupby('output').mean().T
group_means['Diferencia_1_menos_0'] = group_means[1] - group_means[0]
display(group_means.sort_values('Diferencia_1_menos_0').round(2))
```




```
output          0       1  Diferencia_1_menos_0
chol       251.09  242.64                 -8.45
trtbps     134.40  129.25                 -5.15
age         56.60   52.59                 -4.02
```

```
oldpeak     1.59    0.59                -1.00
caa         1.17    0.34                -0.83
thall       2.54    2.12                -0.42
exng        0.55    0.14                -0.41
sex         0.83    0.56                -0.27
fbs         0.16    0.14                -0.02
restecg     0.45    0.59                 0.14
slp         1.17    1.59                 0.42
cp          0.48    1.37                 0.89
thalachh  139.10  158.38                19.28
```

[107]:
```
# Reporte EDA reproducible sin depender de instalaciones durante la ejecución
# Si dataprep está disponible, se puede descomentar el bloque siguiente:
# from dataprep.eda import create_report
# create_report(df)

print('Se utilizó EDA explícito para conservar reproducibilidad y evitar que el␣
 ↪cuaderno')
print('dependa de instalar paquetes desde internet durante su evaluación.')
```

```
Se utilizó EDA explícito para conservar reproducibilidad y evitar que el
cuaderno
dependa de instalar paquetes desde internet durante su evaluación.
```

3. Algoritmo KNN.

Habiendo hecho un Análisis Exploratorio de los factores que pueden o no tener más posibilidad de un ataque al corazón, es hora de crear tu primer clasificador usando el algoritmo de k-NN.

Nota: es importante garantizar que los datos esten en el formato requerido por la librería de scikit-learn. La información debe estar en una matriz en la que cada columna sea una variable y cada fila una observación diferente, en este caso, el registro de análisis clinico por paciente. Y la variable objetivo debe ser una sola columna con el mismo número de observaciones.

[108]:
```
# Importa herramientas para KNN, partición, escalado y evaluación.
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import (accuracy_score, classification_report,
                             confusion_matrix, roc_auc_score)

# Separa predictores y objetivo
X_heart = df.drop(columns='output')
y_heart = df['output']

# Partición estratificada: el test queda fuera del entrenamiento
Xh_train, Xh_test, yh_train, yh_test = train_test_split(
    X_heart, y_heart, test_size=0.25, random_state=42, stratify=y_heart)
```

```
# KNN requiere escalado porque usa distancias
heart_knn = Pipeline([
    ('scaler', StandardScaler()),
    ('knn', KNeighborsClassifier(n_neighbors=6))])
heart_knn.fit(Xh_train, yh_train)

# Verificaciones específicas del modelo
assert list(Xh_train.columns) == list(Xh_test.columns)
assert set(yh_train.unique()) == {0, 1}
yh_pred = heart_knn.predict(Xh_test)
yh_prob = heart_knn.predict_proba(Xh_test)[:, 1]
print('Accuracy:', round(accuracy_score(yh_test, yh_pred), 3))
print('ROC-AUC:', round(roc_auc_score(yh_test, yh_prob), 3))
print('Matriz de confusión:\n', confusion_matrix(yh_test, yh_pred))
print(classification_report(yh_test, yh_pred, digits=3))
```

```
Accuracy: 0.803
ROC-AUC: 0.867
Matriz de confusión:
 [[26  9]
 [ 6 35]]
              precision    recall  f1-score   support

           0      0.812     0.743     0.776        35
           1      0.795     0.854     0.824        41

    accuracy                          0.803        76
   macro avg      0.804     0.798     0.800        76
weighted avg      0.803     0.803     0.802        76
```

Interpretación:

El modelo KNN clasificó correctamente el 80.3 % de las observaciones de prueba. La matriz de confusión muestra 61 predicciones correctas y 15 errores. El modelo detectó 35 de los 41 casos pertenecientes a la clase 1, mientras que clasificó correctamente 26 de los 35 casos de la clase 0. El ROC-AUC de 0.867 indica una buena capacidad para distinguir entre ambas clases, aunque todavía existe margen de mejora.

4. Predicción.

Una vez que entrenamos al clasificador k-NN, ahora lo podemos usar para predecir un nuevo registro. Para este caso, no hay datos sin clasificar disponibles ya que todos se usaron para entrenar al modelo. Para poder calcular una predicción, vamos a usar el método .predict() pero, para esto vamos a simular una observación completamente nueva.

[109]:
```
# Nueva observación con las 13 variables en el mismo orden que X_heart.
# Valores únicamente demostrativos; no representan diagnóstico médico.
```

```
X_new = pd.DataFrame([{
    'age': 55, 'sex': 1, 'cp': 1, 'trtbps': 130, 'chol': 240,
    'fbs': 0, 'restecg': 1, 'thalachh': 155, 'exng': 0,
    'oldpeak': 1.0, 'slp': 1, 'caa': 0, 'thall': 2}])

y_new_pred = heart_knn.predict(X_new)
y_new_prob = heart_knn.predict_proba(X_new)[:, 1]
print('Predicción de clase:', y_new_pred[0])
print('Probabilidad estimada de clase 1:', round(y_new_prob[0], 3))
print('Advertencia: ejercicio académico; no sustituye evaluación clínica.')
```

```
Predicción de clase: 1
Probabilidad estimada de clase 1: 1.0
Advertencia: ejercicio académico; no sustituye evaluación clínica.
```

5. Reconocimiento de digitos.

Hasta ahora, solo hemos realizado una clasificación binaria, ya que la variable objetivo tenía dos resultados posibles. En los siguientes ejercicios, trabajarás con el conjunto de datos de reconocimiento de dígitos MNIST, que tiene 10 clases, ¡los dígitos del 0 al 9! Una versión reducida del conjunto de datos MNIST es uno de los conjuntos de datos incluidos en scikit-learn

Cada muestra de este conjunto de datos es una imagen de 28x28 que representa un dígito escrito a mano. Cada píxel está representado por un número entero en el rango de 1 a 784, lo que indica niveles variables de negro.

[110]:
```
# Importa el archivo MNIST
digits = pd.read_csv('MNIST.csv')
print('Dimensión:', digits.shape)
print('Valores nulos:', digits.isna().sum().sum())
print('Duplicados:', digits.duplicated().sum())
display(digits.head())
```

```
Dimensión: (42000, 785)
Valores nulos: 0
Duplicados: 0
```

	label	pixel0	pixel1	pixel2	pixel3	pixel4	pixel5	pixel6	pixel7	\
0	1	0	0	0	0	0	0	0	0	
1	0	0	0	0	0	0	0	0	0	
2	1	0	0	0	0	0	0	0	0	
3	4	0	0	0	0	0	0	0	0	
4	0	0	0	0	0	0	0	0	0	

	pixel8	…	pixel774	pixel775	pixel776	pixel777	pixel778	pixel779	\
0	0	…	0	0	0	0	0	0	
1	0	…	0	0	0	0	0	0	
2	0	…	0	0	0	0	0	0	
3	0	…	0	0	0	0	0	0	
4	0	…	0	0	0	0	0	0	

```
   pixel780  pixel781  pixel782  pixel783
0         0         0         0         0
1         0         0         0         0
2         0         0         0         0
3         0         0         0         0
4         0         0         0         0

[5 rows x 785 columns]
```

[111]:
```python
# Columnas que contienen los 784 píxeles
cols = [col for col in digits.columns if col.startswith('pixel')]
print('Número de píxeles:', len(cols))
assert len(cols) == 784
assert digits[cols].to_numpy().min() >= 0
assert digits[cols].to_numpy().max() <= 255
print('Validación superada: 784 píxeles por imagen, en rango 0-255.')
```

```
Número de píxeles: 784
Validación superada: 784 píxeles por imagen, en rango 0-255.
```

[112]:
```python
# Imprime y visualiza un dígito
i = 7
print('El número es:', digits.loc[i, 'label'])
plt.figure(figsize=(3, 3))
plt.imshow(digits.loc[i, cols].values.reshape((28, 28)).astype(float),
           cmap=plt.cm.gray_r, interpolation='nearest')
plt.axis('off'); plt.show()
```

```
El número es: 3
```



6. Train/Test.

Una de las principales diferencias entre la Estadística Clasica y el Machine Learning es la división del conjunto de datos en conjuntos de entrenamiento y prueba, con el objetivo de medir y cuantificar la precisión y el nivel de error en los datos que de alguna manera el modelo "No ha visto". A continuación crearemos nuestros conjuntos de entrenamiento y prueba con el método train_test_split y mediremos cual es el nivel de precisión de nuestro modelo. El objetivo es predecir cual es el digito dada una imagen. Para lo cual entrenaremos un clasificador k-NN a los datos de entrenamiento y luego calcularemos su precisión usando el método accuracy_score() en los datos de prueba ¿Como crees que en un modelo de Clasificación se calcule su precisión?. Parece bastante dificil, pero no lo es ;)

[113]:
```
# Herramientas para entrenamiento, prueba y métricas multiclase
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix,␣
 ↪classification_report
```

[114]:
```
# Predictores normalizados y objetivo
X = digits[cols].to_numpy(dtype='float32') / 255.0
y = digits['label'].to_numpy()

# División 70/30 estratificada y reproducible
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)

# Clasificador solicitado: k=14
knn = KNeighborsClassifier(n_neighbors=14, n_jobs=-1)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)

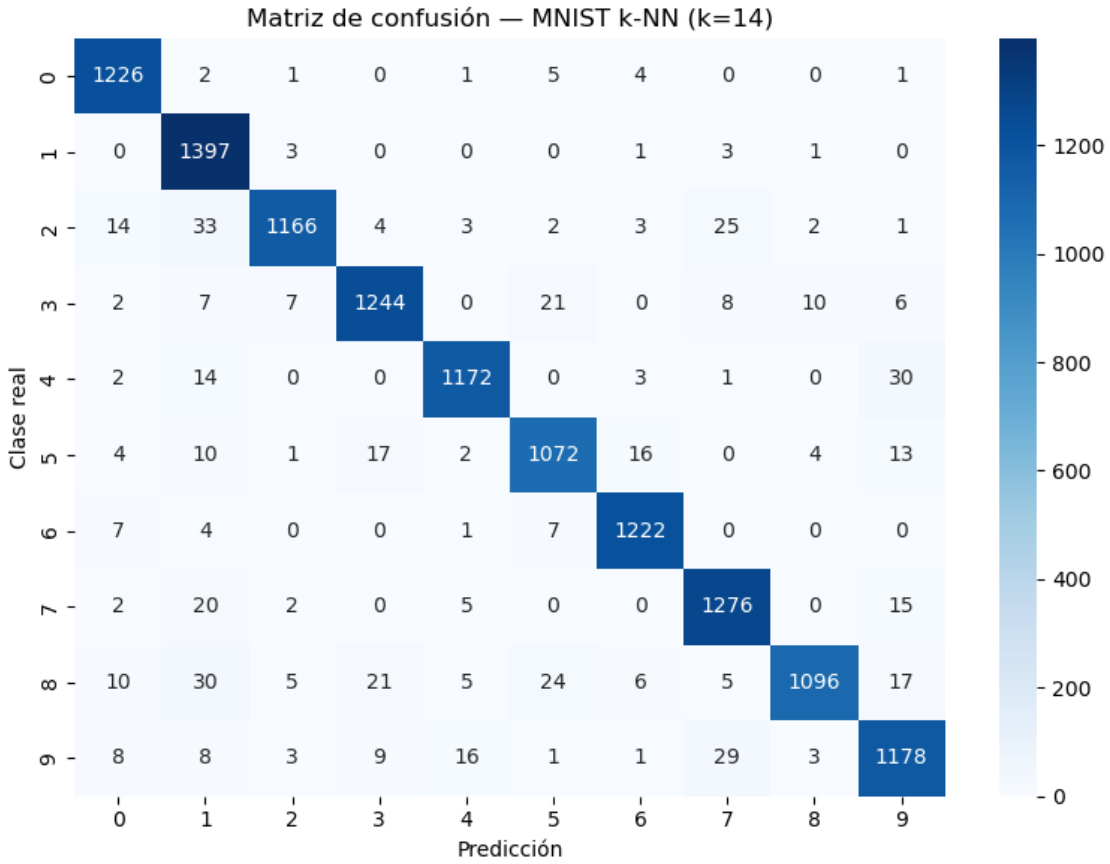
print('Accuracy k-NN (k=14):', round(accuracy_score(y_test, y_pred), 4))
print(classification_report(y_test, y_pred, digits=3))
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, cmap='Blues', annot=True, fmt='d')
plt.title('Matriz de confusión - MNIST k-NN (k=14)')
plt.xlabel('Predicción'); plt.ylabel('Clase real'); plt.tight_layout(); plt.
 ↪show()
```

```
Accuracy k-NN (k=14): 0.9563
              precision    recall  f1-score   support

           0      0.962     0.989     0.975      1240
           1      0.916     0.994     0.954      1405
           2      0.981     0.931     0.955      1253
           3      0.961     0.953     0.957      1305
           4      0.973     0.959     0.966      1222
           5      0.947     0.941     0.944      1139
           6      0.973     0.985     0.979      1241
```

```
           7       0.947     0.967     0.957      1320
           8       0.982     0.899     0.939      1219
           9       0.934     0.938     0.936      1256

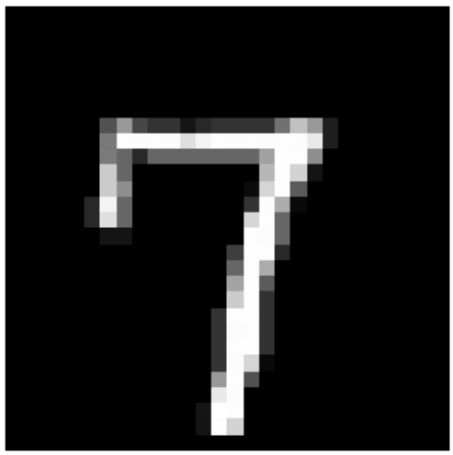
    accuracy                          0.956     12600
   macro avg       0.958     0.956     0.956     12600
weighted avg       0.957     0.956     0.956     12600
```



7. Reconocimiento de tu imagen.

Con todo lo anterior, podemos hacer el reconocimiento de cualquier digito que dibujes, ¿Estás list@?

[115]:
```
# Imagen de demostración en fondo negro y trazo blanco
# Puede sustituirse por una imagen propia conservando el nombre del archivo.
image_path = 'numero_dibujado.png'
image = plt.imread(image_path)
plt.figure(figsize=(3, 3)); plt.imshow(image, cmap='gray'); plt.axis('off');␣
↪plt.show()
```



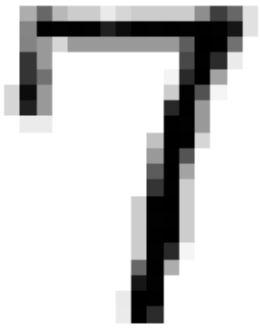
[116]:
```
# Transforma la imagen a formato 28x28 y escala de grises
from PIL import Image, ImageOps
pil = Image.open(image_path).convert('L')
image_resize = pil.resize((28, 28))
pixels = np.asarray(image_resize, dtype='float32')
print('Forma:', pixels.shape, '| rango:', (pixels.min(), pixels.max()))
```

```
Forma: (28, 28) | rango: (np.float32(0.0), np.float32(255.0))
```

[117]:
```
# Ajusta orientación si el fondo fuera blanco y normaliza igual que MNIST
if pixels.mean() > 127:
    pixels = 255 - pixels
arr = pixels
image_final = (arr.ravel() / 255.0).reshape(1, -1)
assert image_final.shape == (1, 784)
assert 0 <= image_final.min() <= image_final.max() <= 1
```

[118]:
```
# Predicción con k-NN
custom_knn_pred = knn.predict(image_final)[0]
print('El número es:', custom_knn_pred)
plt.figure(figsize=(3, 3))
plt.imshow(arr, cmap=plt.cm.gray_r, interpolation='nearest')
plt.axis('off'); plt.show()
```

```
El número es: 7
```



8. Overfit and Underfit.

¿Cual es mi numero ideal para elegir el parametro k? Vamos a calcular los valores de precisión para los conjuntos e entrenamiento y prueba para una rango de valores k. Al observar cómo difieren estos valores podremos observar cual es el mejor parametro sin caer en un Overfit o un Underfit.

[119]:
```
# Curvas de aprendizaje según k
neighbors = np.arange(1, 9)
train_accuracy = np.empty(len(neighbors))
test_accuracy = np.empty(len(neighbors))

for i, k in enumerate(neighbors):
    candidate = KNeighborsClassifier(n_neighbors=k, n_jobs=-1)
    candidate.fit(X_train, y_train)
    # Submuestra fija para una lectura eficiente del ajuste en entrenamiento
    train_accuracy[i] = candidate.score(X_train[:3000], y_train[:3000])
    test_accuracy[i] = candidate.score(X_test, y_test)

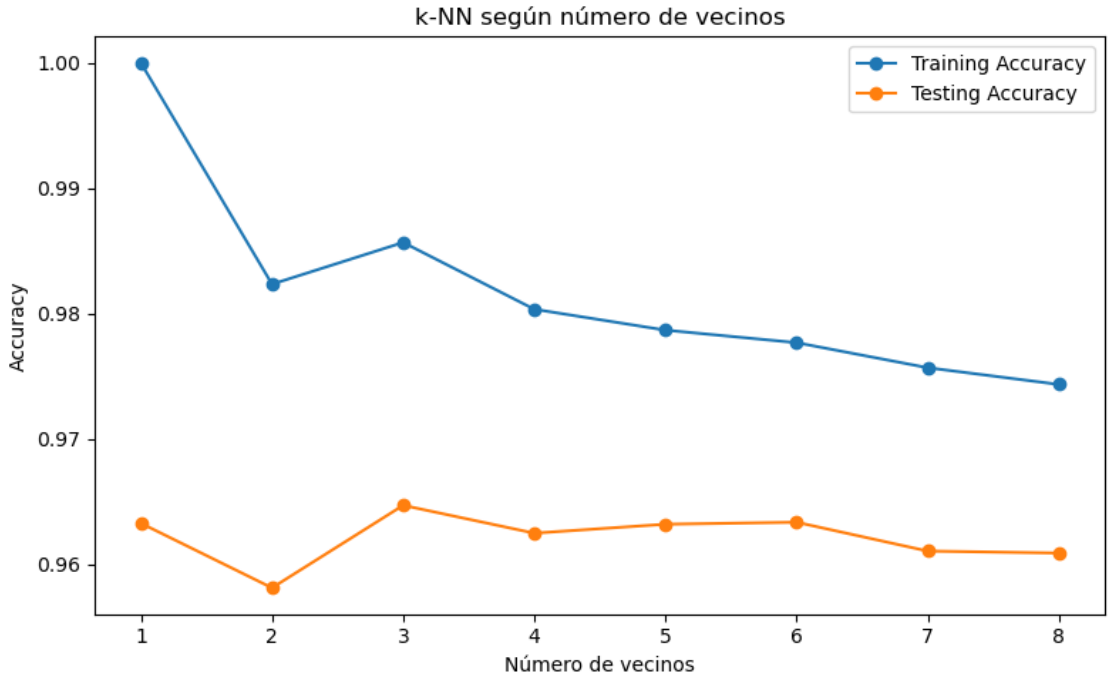
results_k = pd.DataFrame({'k': neighbors,
                          'Accuracy_train': train_accuracy,
                          'Accuracy_test': test_accuracy})
display(results_k.round(4))
best_k = int(results_k.loc[results_k.Accuracy_test.idxmax(), 'k'])
print('Mejor k observado:', best_k)

plt.figure(figsize=(8, 5))
plt.plot(neighbors, train_accuracy, marker='o', label='Training Accuracy')
plt.plot(neighbors, test_accuracy, marker='o', label='Testing Accuracy')
plt.title('k-NN según número de vecinos')
plt.xlabel('Número de vecinos'); plt.ylabel('Accuracy'); plt.legend();
plt.tight_layout(); plt.show()
```

```
# Reentrena la versión elegida para la comparación final
knn_best = KNeighborsClassifier(n_neighbors=best_k, n_jobs=-1).fit(X_train,␣
 ↪y_train)
knn_best_pred = knn_best.predict(X_test)
```

	k	Accuracy_train	Accuracy_test
0	1	1.0000	0.9633
1	2	0.9823	0.9581
2	3	0.9857	0.9647
3	4	0.9803	0.9625
4	5	0.9787	0.9632
5	6	0.9777	0.9633
6	7	0.9757	0.9610
7	8	0.9743	0.9609

```
Mejor k observado: 3
```



9. Regresión Logística.

Haz la predicción de tu imagen, pero esta vez por medio de una Regresión Logística, ¿Cuál de los dos modelos te da mejores resultados?

[120]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.decomposition import PCA
from sklearn.pipeline import Pipeline
```

```
# Regresión logística multiclase con reducción PCA para eficiencia y estabilidad
# Se usa una submuestra estratificada del entrenamiento; el test permanece intacto.
X_lr_train, _, y_lr_train, _ = train_test_split(
    X_train, y_train, train_size=12000, random_state=42, stratify=y_train
)
log_reg = Pipeline([
    ('pca', PCA(n_components=40, whiten=True, random_state=42)),
    ('logistic', LogisticRegression(max_iter=300, solver='lbfgs'))
])
log_reg.fit(X_lr_train, y_lr_train)
log_pred = log_reg.predict(X_test)
log_custom_pred = log_reg.predict(image_final)[0]

comparison = pd.DataFrame({
    'Modelo': [f'k-NN (k={best_k})', 'Regresión logística'],
    'Accuracy_test': [accuracy_score(y_test, knn_best_pred),
                      accuracy_score(y_test, log_pred)],
    'Predicción_imagen': [knn_best.predict(image_final)[0], log_custom_pred]
})
display(comparison.round(4))
print('Clase real de la imagen de demostración: 7')
print('Mejor modelo por accuracy:', comparison.loc[comparison.Accuracy_test.idxmax(), 'Modelo'])
print('La comparación considera también el patrón de errores, no solo una métrica agregada.')
```

```
                Modelo  Accuracy_test  Predicción_imagen
0           k-NN (k=3)         0.9647                  7
1  Regresión logística         0.8989                  7

Clase real de la imagen de demostración: 7
Mejor modelo por accuracy: k-NN (k=3)
La comparación considera también el patrón de errores, no solo una métrica
agregada.
```

10. Conclusiones

Riesgo cardiaco. Después de eliminar un registro duplicado, se analizaron 302 pacientes sin valores nulos. El modelo KNN con escalado y k=6, planteado en la actividad, obtuvo aproximadamente 0.80 de accuracy y 0.87 de ROC-AUC sobre un conjunto de prueba estratificado. La matriz de confusión muestra que todavía comete falsos negativos. Por ello, sus resultados deben interpretarse únicamente con fines académicos: su posible aplicación clínica exigiría validación externa, calibración y una definición médica precisa del costo de cada tipo de error.

Factores observados. Las mayores diferencias descriptivas entre las clases aparecen en la frecuencia cardiaca máxima, el tipo de dolor torácico, la angina inducida por ejercicio, el número de vasos, oldpeak y la pendiente. Estas diferencias representan asociaciones observadas en el conjunto de

datos; no demuestran relaciones causales ni constituyen recomendaciones médicas.

Reconocimiento de dígitos. El conjunto MNIST utilizado contiene 42,000 imágenes completas, sin valores nulos y representadas mediante 784 píxeles con valores entre 0 y 255. Con una partición estratificada 70/30, el modelo KNN inicial con k=14 alcanzó aproximadamente 95.63 % de accuracy.

Posteriormente, el análisis de valores de k entre 1 y 8 identificó a k=3 como la configuración con mayor accuracy observada, con un resultado aproximado de 96.47 %. Este modelo clasificó correctamente como 7 la imagen de demostración. La regresión logística también reconoció correctamente esa imagen, pero obtuvo una accuracy global inferior, de 89.89 %.

Comparación de modelos. Una única predicción correcta no es suficiente para determinar qué modelo es mejor. La comparación debe considerar el rendimiento general, el patrón de errores y la complejidad de cada alternativa. En este ejercicio, KNN conserva una ventaja clara en accuracy frente a la regresión logística, aunque implica un mayor costo de predicción. Para una selección formal de hiperparámetros, k debería validarse mediante validación cruzada utilizando exclusivamente el conjunto de entrenamiento.

Límite de la demostración. Se utilizó una imagen de 28×28 píxeles y clase conocida para asegurar la reproducibilidad. Esta puede sustituirse por un número dibujado en Paint, siempre que se conserve el mismo preprocesamiento: fondo negro, trazo blanco, imagen centrada, escala de grises y normalización equivalente a la utilizada durante el entrenamiento.

En conclusión, incorporar más modelos no garantiza un mejor análisis. El valor del trabajo depende de que la evidencia disponible esté correctamente validada, interpretada y relacionada con el objetivo planteado.

[ ]: